

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	ATHARSH S U	
Register Number	24011101008	
Roll Number	24110049	
Class	AI-DS 'A'	

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap

- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code (GitHub Repo)

Repository Link:

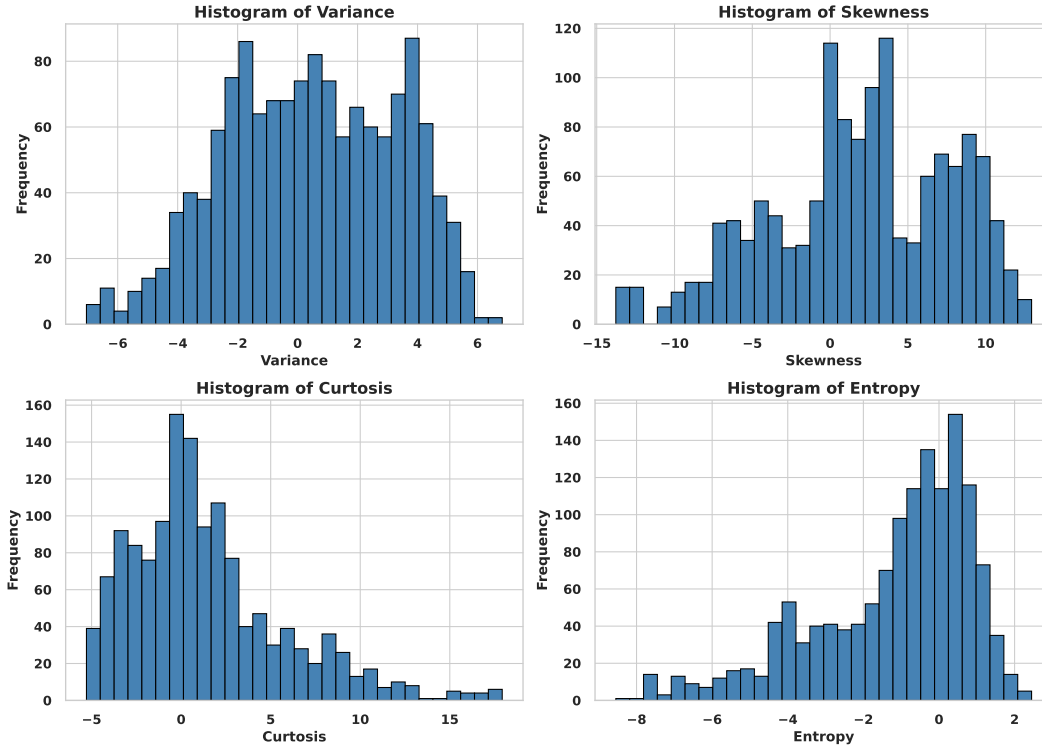
<https://github.com/Atharsh2910/Perceptron-Lab1>

7. Experimental Results

- Accuracy = 0.9891
- Precision = 0.9917
- Recall = 0.9836
- F1-score = 0.9877
- Confusion Matrix = $\begin{bmatrix} 152 & 1 \\ 2 & 120 \end{bmatrix}$

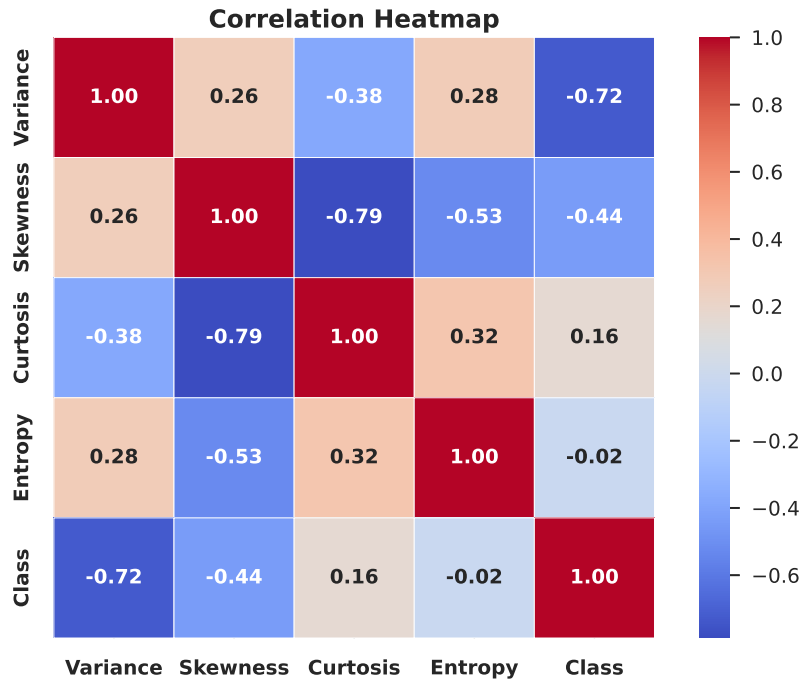
8. Generated Plots

8.1 Feature Histograms



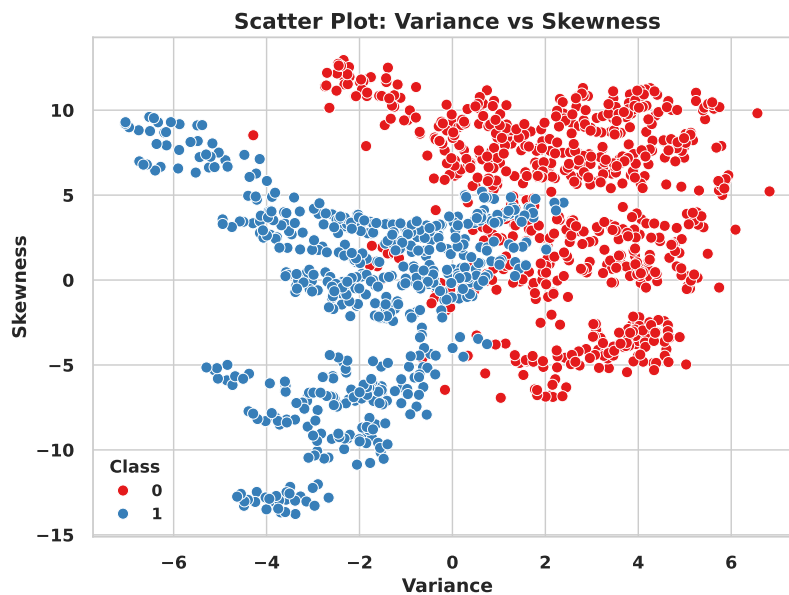
Inference: The variance appears to be normally distributed whereas skewness, kurtosis and entropy are skewed towards the sides. This shows that the datasets have the points that can be separated as classes.

8.2 Correlation Heatmap



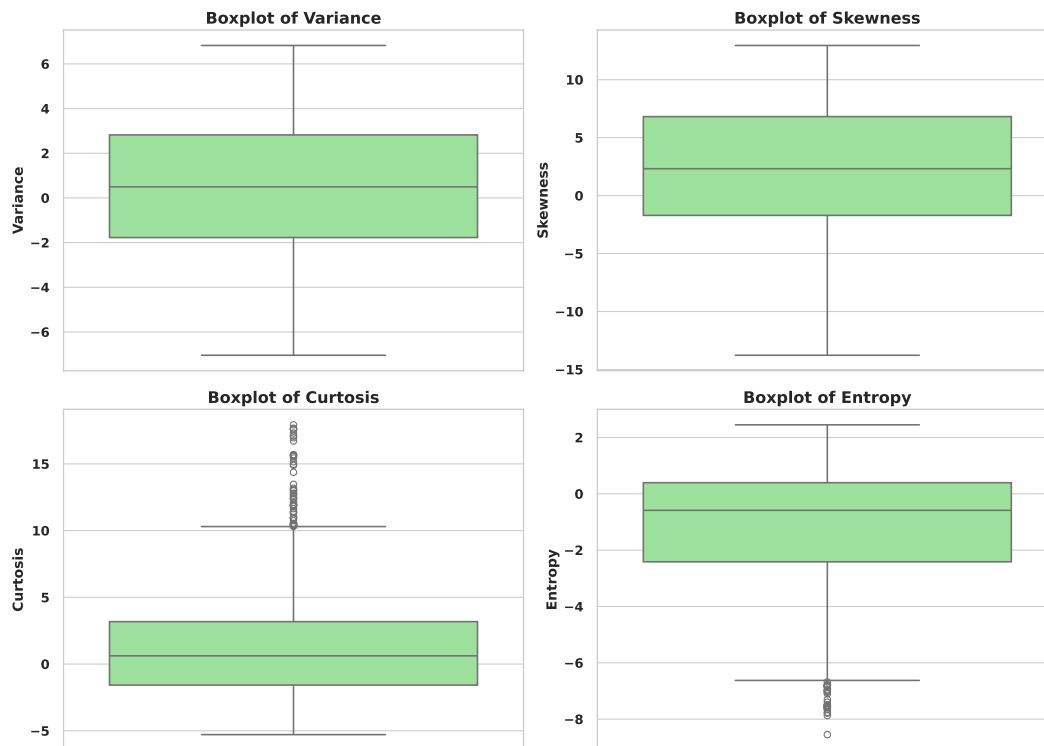
Inference: The correlation between the attributes are weaker and shows no strong correlation. Thus all attributes has to be used for training.

8.3 Scatter Plot



Inference: The scatter plot for two features has data points that overlap each other. Thus for classification purpose, we have to use all the features for proper linear separation.

8.4 Boxplots



Inference: Entropy and Kurtosis has many outlier points that lie beyond the whisker, in the plot. This proves the skewness captured in the histogram.

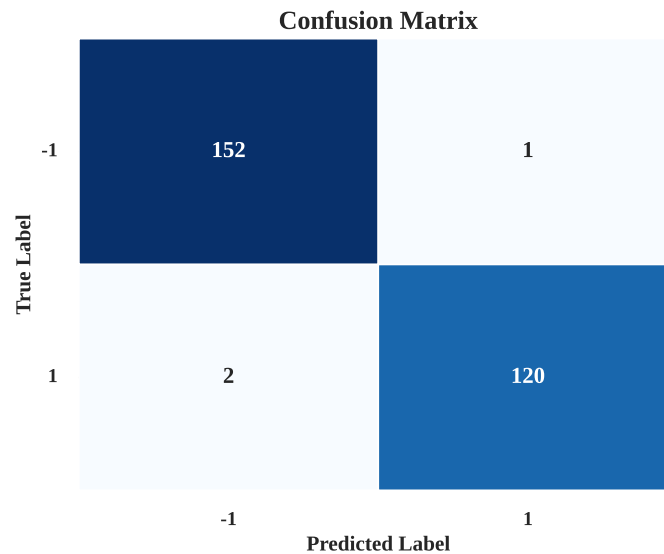
8.5 Training Error vs Epoch



Inference: The training error increases drastically beyond Three epochs. This suggests that the Perceptron has generalised the data points in early epochs. The convergence occurs at around 3

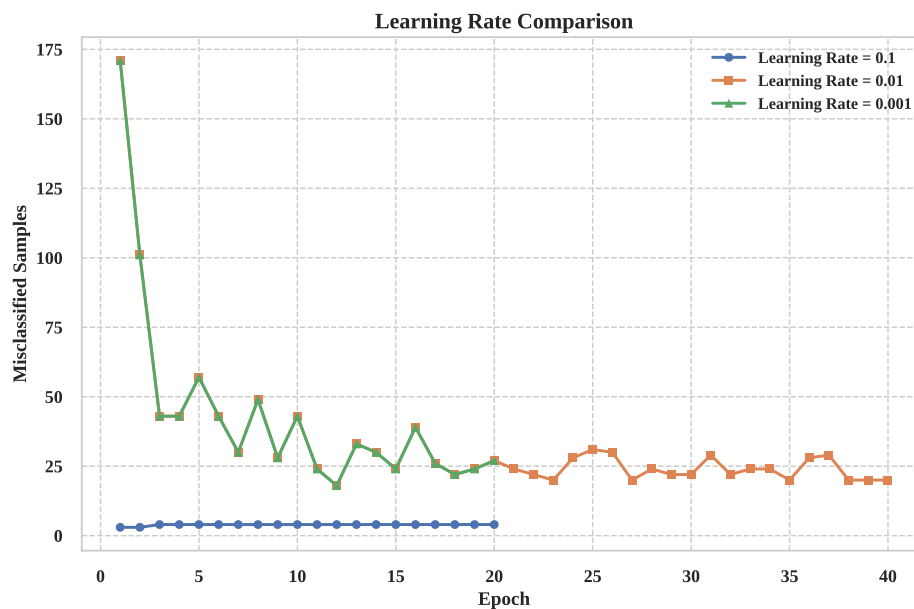
epochs.

8.6 Confusion Matrix



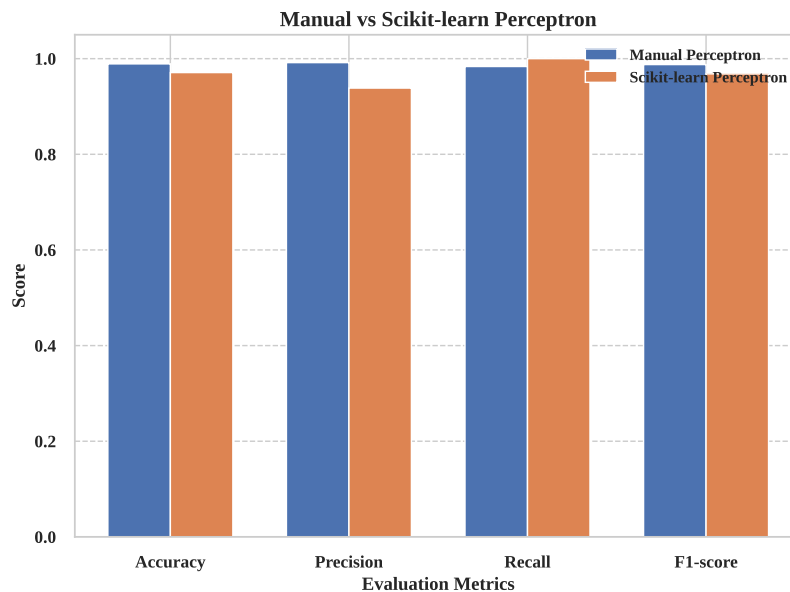
Inference: Out of 275 test samples, 152 original and 120 duplicate notes were correctly classified, with only 1 false positive and 2 false negatives. This suggests that single layer perceptron provides high accuracy classification for this dataset.

8.7 Learning Rate Comparison



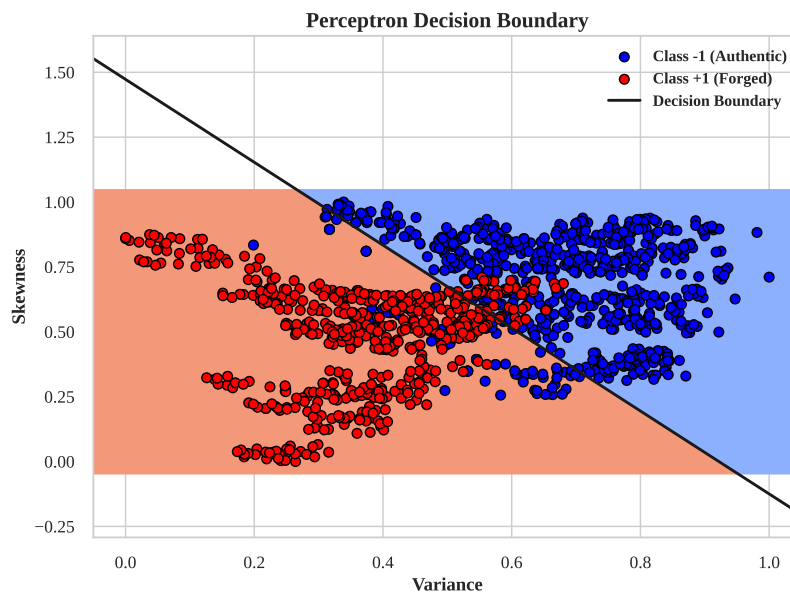
Inference: Learning rate of 0.1 provides the best convergence and maintains the same accuracy for all epochs. For learning rates 0.01 and 0.001, the accuracy drastically increases beyond the 10th epoch, where the convergence occurs.

8.8 Manual vs Scikit-learn Perceptron



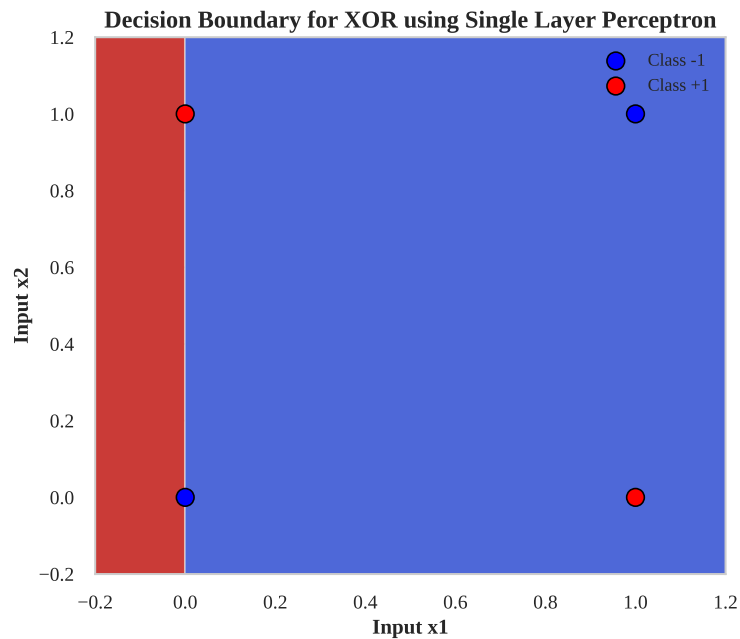
Inference: Both Manual and Scikit-Learn version of perceptron provides almost equal classification accuracy, with manual perceptron slightly outperforming the scikit learn version.

8.9 Decision Boundary (Skewness & Kurtosis)



Inference: With just two attributes, the decision boundary could not be defined well, with many points overlapping near the maximum margin separation line. This suggests that all four attributes are equally important for the classification.

8.10 XOR Decision Boundary



Inference: Single Layer Perceptron cannot work on XOR gate as it is proved that a single line of separation could not be obtained. This shows the limitation of a SLP, to linear separability.

9. Performance Tables

9.1 Training Summary

Dataset Size	1372 items
Train/Test Split	80:20 (1097 train / 275 test)
Learning Rates Studied	0.1, 0.01, 0.001
Epochs	50
Final Weights	$[-0.1628, -0.1898, -0.1866, 0.0013]$
Final Bias	0.2300
Accuracy	0.9891
Precision	0.9917
Recall	0.9836
F1-score	0.9877

9.2 Epoch-wise Learning ($\eta = 0.01$)

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	171	-0.0679	-0.0462	-0.0590	0.0221	0.07
5	57	-0.0988	-0.0983	-0.0984	0.0111	0.13
10	43	-0.1114	-0.1198	-0.1274	0.0099	0.16
15	24	-0.1169	-0.1380	-0.1421	0.0009	0.17
20	27	-0.1286	-0.1474	-0.1441	-0.0067	0.19
25	31	-0.1319	-0.1506	-0.1608	0.0039	0.20
30	22	-0.1370	-0.1639	-0.1710	-0.0018	0.20
35	20	-0.1450	-0.1739	-0.1713	-0.0013	0.21
40	20	-0.1545	-0.1740	-0.1750	-0.0051	0.22
45	24	-0.1617	-0.1763	-0.1817	-0.0022	0.23
50	20	-0.1628	-0.1898	-0.1866	0.0013	0.23

9.3 Confusion Matrix

	Predicted -1	Predicted +1
Actual -1	152 (TN)	1 (FP)
Actual +1	2 (FN)	120 (TP)

10. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
 1. Step Activation Function provides binary output, making it suitable for simple problems, while the Sigmoid Activation Function produces values between 0 and 1, allowing the model to learn from the data over iterations.
 2. Step Activation Function is not used in modern deep learning because it cannot be differentiated therefore limiting the backpropagation.
2. Compare the implementation with Scikit-learn's Perceptron.

Model	Accuracy	Precision	Recall	F1-score
Manual Perceptron	0.9891	0.9917	0.9836	0.9877
Scikit-learn Perceptron	0.9709	0.9385	1.0000	0.9683

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

Learning Rate	Epochs Run	Final Misclassified	Final Bias
0.1	40	20	2.2000
0.01	50	20	0.2300
0.001	20	27	0.0190

4. Explain why a Single Layer Perceptron cannot solve the XOR problem. A single layer perceptron can handle only linear separation cases. The perceptron cannot derive a straight line that separates the XOR classes, the decision boundary misclassifies at least one of the points, marking it as a limitation of SLP.
5. Study the effect of feature normalization on model convergence.
 1. Feature normalization improves convergence by normalizing all data points to a similar range, allowing the model to update weights more efficiently and converge faster.
 2. Without normalization, larger values can drag the convergence to one direction, thereby slowing down convergence.

11. Conclusion

- The model achieved **98.91% accuracy**, **99.17% precision**, **98.36% recall**, and an **F1-score of 98.77%** on the test set, correctly classifying 272 of 275 test samples.
- Both the models, manual and scikit learn perceptron, achieved almost similar accuracy (98.91% vs. 97.09% accuracy).
- This shows that Single Layer Perceptron can be used for binary classifications efficiently.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.